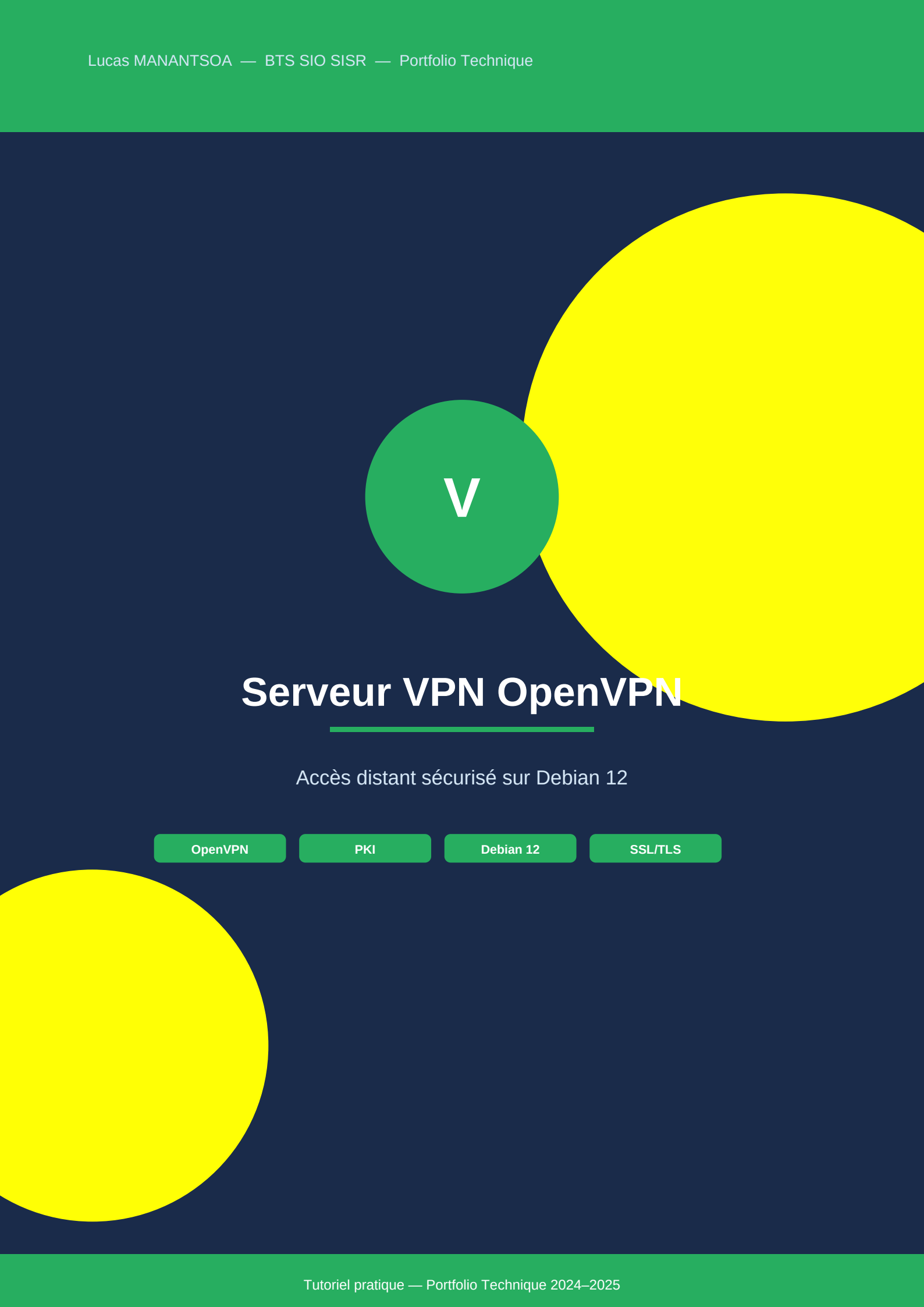




V

Serveur VPN OpenVPN

Accès distant sécurisé sur Debian 12

OpenVPN

PKI

Debian 12

SSL/TLS

1. Introduction et objectif

Dans ce tuto, on va mettre en place un serveur VPN avec **OpenVPN** sur une machine Debian 12. L'idée c'est de permettre à n'importe quel collaborateur de se connecter au réseau interne depuis chez lui, de façon totalement chiffrée et sécurisée. On utilise une infrastructure à clé publique (PKI) avec Easy-RSA pour générer les certificats — c'est ce qui garantit que seul quelqu'un avec un cert valide peut se connecter.

i

Prérequis : Debian 12, accès root, une IP publique (ou IP WAN accessible). Dans notre cas : IP LAN 10.7.255.30, IP WAN 10.7.250.30.

2. Installation d'OpenVPN et Easy-RSA

On commence par mettre à jour le système et installer les paquets nécessaires. Easy-RSA va nous servir à créer notre propre autorité de certification.

```
1 # Mise à jour du système
2 sudo apt update && sudo apt upgrade -y
3
4 # Installation d'OpenVPN et Easy-RSA
5 sudo apt install openvpn easy-rsa -y
6
7 # Vérification des versions
8 openvpn --version
9 ls /usr/share/easy-rsa/
```

3. Initialisation de la PKI avec Easy-RSA

La PKI (Public Key Infrastructure) c'est le coeur du système. On va créer une autorité de certification interne qui va signer tous nos certificats. Copie Easy-RSA dans un répertoire dédié.

1 Préparer l'environnement Easy-RSA

```
1 # Copier Easy-RSA dans /etc/openvpn
2 sudo make-cadir /etc/openvpn/easy-rsa
3 cd /etc/openvpn/easy-rsa
4
5 # Initialiser la PKI
6 sudo ./easyrsa init-pki
```

2 Créer l'Autorité de Certification (CA)

Le CA c'est la racine de confiance de toute ton infrastructure. Son certificat (ca.crt) sera distribué à tous les clients pour qu'ils puissent vérifier l'authenticité du serveur.

```
1 # Génération du CA (sans mot de passe pour simplifier)
2 sudo ./easyrsa build-ca nopass
3
4 # Il te demande un Common Name – entre par exemple : lmstreetrider-CA
5 # Le certificat CA sera dans : /etc/openvpn/easy-rsa/pki/ca.crt
```

3 Générer le certificat et la clé du serveur

```
1 # Générer le certificat serveur (sans passphrase)
2 sudo ./easyrsa build-server-full server nopass
3
4 # Générer les paramètres Diffie-Hellman (peut prendre quelques minutes)
5 sudo ./easyrsa gen-dh
6
7 # Générer la clé TLS-Auth (protection contre les attaques DoS)
8 sudo openvpn --genkey secret /etc/openvpn/easy-rsa/pki/ta.key
```

4 Générer les certificats clients

Pour chaque utilisateur, on génère un certificat unique. Ici on crée des certs pour les 6 collaborateurs de LMStreetRider.

```
1 # Un cert par utilisateur
2 sudo ./easyrsa build-client-full ManantsoaL nopass
3 sudo ./easyrsa build-client-full DupontD nopass
4 sudo ./easyrsa build-client-full DuboisM nopass
5 sudo ./easyrsa build-client-full BernardC nopass
6 sudo ./easyrsa build-client-full ChevalierN nopass
7 sudo ./easyrsa build-client-full AnaisC nopass
```

5 Copier les fichiers vers le répertoire OpenVPN

```
1 sudo cp pki/ca.crt /etc/openvpn/server/
2 sudo cp pki/issued/server.crt /etc/openvpn/server/
3 sudo cp pki/private/server.key /etc/openvpn/server/
4 sudo cp pki/dh.pem /etc/openvpn/server/
5 sudo cp pki/ta.key /etc/openvpn/server/
```

4. Configuration du serveur OpenVPN

C'est le fichier de config principal. Il définit tout : le réseau du tunnel, les routes à pousser vers les clients, le chiffrement, etc.

```
1 sudo nano /etc/openvpn/server/server.conf
```

Voici le contenu complet du fichier de configuration :

```
1 port 1194
2 proto udp
3 dev tun
4
5 # Certificats PKI
6 ca /etc/openvpn/server/ca.crt
7 cert /etc/openvpn/server/server.crt
8 key /etc/openvpn/server/server.key
9 dh /etc/openvpn/server/dh.pem
10 tls-auth /etc/openvpn/server/ta.key 0
11
12 # Réseau du tunnel VPN
13 server 10.8.0.0 255.255.255.0
14 ifconfig-pool-persist /var/log/openvpn/ipp.txt
15
16 # Routes poussées vers les clients (accès aux VLANs internes)
17 push "route 10.7.250.0 255.255.255.0"
18 push "route 10.7.255.0 255.255.255.0"
19 push "dhcp-option DNS 172.16.114.44"
20
21 # Paramètres de connexion
22 keepalive 10 120
23 cipher AES-256-GCM
24 auth SHA256
25 compress lz4-v2
26 push "compress lz4-v2"
27
28 persist-key
29 persist-tun
30 user nobody
31 group nogroup
32
33 # Journalisation
34 status /var/log/openvpn/openvpn-status.log
```

```
1 log-append /var/log/openvpn/openvpn.log
2 verb 3
```

5. Activation du forwarding IP

Pour que le serveur puisse router le trafic entre le tunnel VPN et le réseau interne, il faut activer le forwarding IP au niveau du noyau Linux.

```
1 # Activer temporairement (immédiat)
2 sudo sysctl -w net.ipv4.ip_forward=1
3
4 # Rendre permanent (après reboot)
5 sudo nano /etc/sysctl.conf
6 # → Décommenter ou ajouter la ligne :
7 # net.ipv4.ip_forward=1
8
9 # Appliquer sans redémarrer
10 sudo sysctl -p
```

6. Démarrage et activation du service

```
1 # Démarrer le service OpenVPN
2 sudo systemctl start openvpn-server@server
3
4 # Activer le démarrage automatique
5 sudo systemctl enable openvpn-server@server
6
7 # Vérifier le statut
8 sudo systemctl status openvpn-server@server
9
10 # Vérifier que l'interface tun0 est créée
11 ip addr show tun0
```



Si le service démarre bien, tu dois voir l'interface tun0 avec l'IP 10.8.0.1 dans la sortie de ip addr.

7. Création du fichier client (.ovpn)

Chaque client reçoit un fichier .ovpn qui contient tout ce dont il a besoin pour se connecter. On peut intégrer les certificats directement dans le fichier.

```
1  # Script pour générer le .ovpn d'un client
2  #!/bin/bash
3  CLIENT=$1
4  PKI=/etc/openvpn/easy-rsa/pki
5
6  cat > /root/${CLIENT}.ovpn << EOF
7  client
8  dev tun
9  proto udp
10 remote 10.7.250.30 1194
11 resolv-retry infinite
12 nobind
13 cipher AES-256-GCM
14 auth SHA256
15 key-direction 1
16 EOF
17
18 echo '<ca>' >> /root/${CLIENT}.ovpn
19 cat $PKI/ca.crt >> /root/${CLIENT}.ovpn
20 echo '</ca>' >> /root/${CLIENT}.ovpn
21
22 echo '<cert>' >> /root/${CLIENT}.ovpn
23 cat $PKI/issued/${CLIENT}.crt >> /root/${CLIENT}.ovpn
24 echo '</cert>' >> /root/${CLIENT}.ovpn
25
26 echo '<key>' >> /root/${CLIENT}.ovpn
27 cat $PKI/private/${CLIENT}.key >> /root/${CLIENT}.ovpn
28 echo '</key>' >> /root/${CLIENT}.ovpn
29
30 echo '<tls-auth>' >> /root/${CLIENT}.ovpn
31 cat $PKI/ta.key >> /root/${CLIENT}.ovpn
32 echo '</tls-auth>' >> /root/${CLIENT}.ovpn
```

```
1  # Utilisation du script
2  sudo bash gen_client.sh ManantsoaL
3  # → Génère /root/ManantsoaL.ovpn à envoyer à l'utilisateur
```

8. Commandes de diagnostic

Quelques commandes utiles pour surveiller et déboguer le serveur VPN :

```
1 # Voir les clients actuellement connectés
2 sudo cat /var/log/openvpn/openvpn-status.log
3
4 # Suivre les logs en temps réel
5 sudo journalctl -u openvpn-server@server -f
6
7 # Voir les routes créées par OpenVPN
8 ip route | grep tun
9
10 # Tester la connectivité depuis un client connecté
11 ping 10.7.255.254 # SW3
12 ping 172.16.114.44 # SRV-LMSR
```



En cas d'échec de connexion, vérifier que le port UDP 1194 est ouvert dans le pare-feu (ufw allow 1194/udp).

9. Tableau récapitulatif

Réseau tunnel	10.8.0.0/24 — IP serveur : 10.8.0.1
Port	UDP/1194
Chiffrement	AES-256-GCM + SHA256
PKI	Easy-RSA 3 — CA interne lmstreetrider
Config serveur	/etc/openvpn/server/server.conf
Logs	/var/log/openvpn/openvpn.log
Démarrage auto	systemctl enable openvpn-server@server